Project Management Document - Real-Time Health Telemetry Platform

Repository: github.khoury.northeastern.edu:neilisrani/Final_Project_Neil_Israni

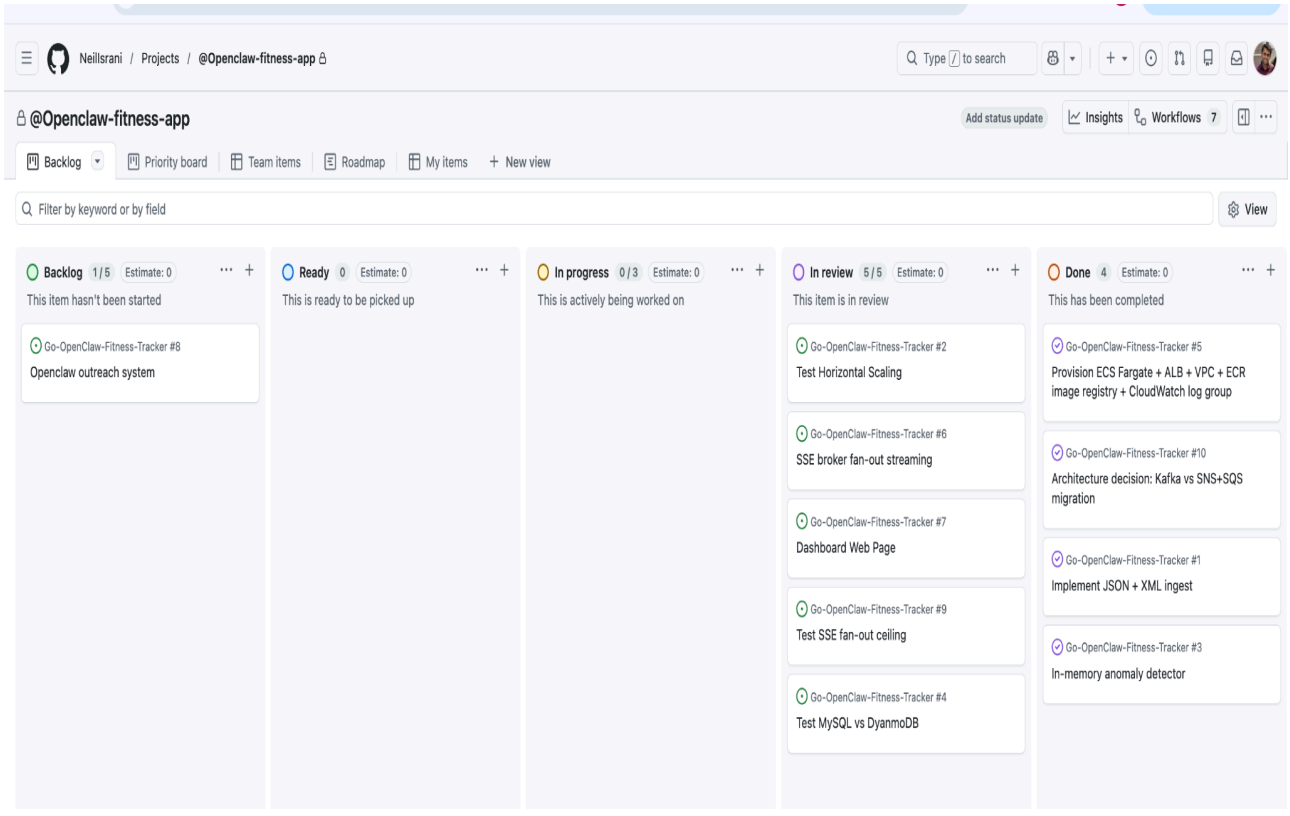
Overview:

This is a real-time telemetry backend that ingests Apple Watch data, detects anomalies using a sliding-window z-score, and streams live metrics to a browser dashboard. I organised the project using a lightweight, code-centric workflow built around GitHub and GitHub Projects. All work is tracked as issues representing discrete features, infrastructure tasks, experiments, and deliverables, labelled by functional area (infrastructure, backend/core, agent, test/experiment) and progressing through a simple Backlog → Ready → In Progress → In Review → Done lifecycle.

Project Management:

I used GitHub Projects to map the project management of this scalable application.

For the HW9 Project lift-off, this is what the implementation looked like:



From there, tickets to implement in the next few weeks were:

Feature/ Decision	Options	Chosen	Rationale
Rolling window state location	Redis, in-memory	In-memory	Establish latency ceiling before adding network round-trips
When to add persistence	Immediately, after Exp 2	After Exp 2	Need baseline before knowing what overhead is acceptable
Streaming backbone	Kafka, SNS+SQS	SQS	AWS-native, no cluster overhead, load does not need Kafka ordering
Queue implementation	SQS, ChannelQueue	ChannelQueue (local) / SQS (prod)	Same interface, env-var toggle — no code change to switch
Persistence backend	DynamoDB, MySQL/RDS	DynamoDB (planned)	Partition-key-per-user access pattern; HW8 implementation directly reusable
OpenClaw agent	Keep, remove	Removed	Prompt-only file, no Go coupling, out of scope for submission
SSE overload handling	Queue, bulkhead, drop	Bulkhead (503 at 500 clients)	Fast-fail is preferable to silent ALB timeout; converts 2.1% failure to explicit rejection
Fault tolerance approach	SQS alone, DynamoDB alone, both	SQS for in-flight + DynamoDB for history (planned)	Complementary — SQS prevents mid-queue loss on crash, DynamoDB prevents history loss on restart

Progress:

Phase 1 (2/18/2026 to 3/3/2026)— Go-based Pipeline and Application Builds

Built the full Go backend from scratch. This included the data models (HealthRecord, AnomalyEvent, StatsResponse), the in-memory time-series store (store.go — sorted slices per metric type, binary-search insert at O(log n), range queries via binary search, protected by sync.RWMutex), and the streaming XML parser (parser.go — xml.Decoder token-by-token over Apple Health export files that can exceed 1 GB, piping records into a buffered channel). Added the sliding-window z-score anomaly detector (anomaly.go — 200-sample window with O(1) running sum/sumSq accumulators, z≥2.0 warning, z≥3.0 critical, 10-sample warm-up minimum).

Completed the SSE broker (sse.go — goroutine-per-client fan-out, 64-slot per-client buffer, non-blocking select/default drops slow clients rather than blocking the write path). Wired everything together in main.go and handlers.go with all routes operational.

Terraform was initialised by following the HW5 module structure (network, ecr, ecs, logging) and updating variable defaults for the new service name. It hosted the ingest of real Apple Watch data to verify the end-to-end pipeline.

Problem encountered: Apple Health XML and JSON exports from different iPhone versions use inconsistent date formats. Fixed by iterating over a list of known formats at parse time until one succeeds.

Phase 2 (3/2/2026 to 3/15/2026) — ALB, Terraform Infrastructure & Dashboard

The ALB was added to Terraform, replacing direct ECS task public IP access. This required:

Adding the alb module with listener, target group, and /health check configuration
Creating a custom VPC with dual public subnets — the AWS Academy Learner Lab environment does not provision a default VPC
Updating the ECS security group to only accept traffic from the ALB security group rather than 0.0.0.0/0

Substituting LabRole for the standard ECS task execution role (Academy environments restrict IAM role creation)

The dashboard was rebuilt with live Chart.js charts for heart rate, steps, and calories, an anomaly feed, and an EventSource connection to /stream.

Problems encountered: LabRole substitution: ECS task definition requires an execution role ARN. Academy LabRole does not have the standard ECS trust relationship. Resolution: referenced LabRole ARN directly in the task definition rather than creating a new role.

Phase 3 (3/15/2026 to 3/30/2026) — Experiments & Improving Application

Developed three tests and ran against the live AWS infrastructure:

Experiment 1 (tests/locust/exp1_scaling.py): IngestUser (weight 7, POST synthetic readings every ~1s) and ReadUser (weight 3, GET metrics/stats/anomalies every 2–4s). Run at 90 users with ECS task count changed via terraform apply -var="ecs_count=N" between runs.

Experiment 2 (tests/run_tests.go): Sequential Go script — 50 ingest batches, 50 metric queries, 50 stats aggregations. Sequential by design: no queuing effects, each measurement is isolated. The -backend none|dynamo|mysql flag was scaffolded for future comparison runs.

Experiment 3 (tests/locust/exp3_sse_fanout.py): SSEClientUser holds /stream open for 20–40s using iter_content; AnomalyDriverUser POSTs extreme heart rates (5, 8, 220, 225 bpm) at 4–8 req/s to continuously trigger z-score alerts. Rolling window primed with 50 normal readings before injection.

Problems encountered: These results identified two clear gaps to address: the synchronous ingest handler was blocking on store writes and SSE broadcast under load, and the SSE broker had no connection limit to prevent unbounded fan-out cost. The goal was to improve the scalability of the application, so I decided to add to the complexity in the next week.

Phase 4 (3/20/2026 to 4/18/2026) — Improving Scalability: Async Queue + Bulkhead + Test improvement

I switched from AWS Academy Learner Lab to Innovative Sandbox for this phase, enabling real SQS access and removing the credential expiry limitations of the Learner Lab.

The primary architectural addition this week was an async ingest pipeline, motivated directly by Experiment 1 and 3 findings. Rather than processing each record synchronously in the HTTP request goroutine, ingest handlers now enqueue the record and return 202 Accepted immediately. A background worker pool drains the queue and handles Store.Add → Detector.Detect → Broker.Broadcast independently of the HTTP path. Two queue backends were implemented behind a common interface: an SQS-backed queue for production (activated via SQS_QUEUE_URL env var) and an in-process channel queue as a local fallback — identical semantics, no code change to switch between them.

Architecture decision — SQS over Kafka: Kafka requires a managed cluster (MSK) or self-hosted infrastructure. SQS was already familiar from coursework and integrated cleanly with the existing ECS task role. DynamoDB was also evaluated as a complementary layer for historical durability and to solve the split-brain problem from Experiment 1B (each task holding an independent rolling window). It was not implemented within this scope — the primary goal was measuring distributed systems tradeoffs, and the analytical case for DynamoDB is stronger

when backed by the in-memory baseline Experiment 2 now provides. That baseline is the reference point against which DynamoDB's ~1–5ms per-write overhead would be compared in a follow-on implementation.

SSE connection bulkhead — The Experiment 3 results drove a concrete architectural addition. The SSE broker's Broadcast method iterates every connected client goroutine on every event — O(n) cost per anomaly. With AnomalyDriverUser injecting extreme readings at 4–8 req/s, CPU hit 99% with only ~85 concurrent clients. Allowing connections to grow unboundedly would push broadcast cost past any CPU budget the Fargate task can sustain.

The bulkhead addresses this by rejecting /stream connections beyond a fixed limit with an immediate 503 before the broker is ever involved — no goroutine allocated, no subscription registered, no future broadcast cost incurred. This also converts the original Experiment 3 failure mode (connections silently hanging until the ALB's 60-second idle timeout killed them, producing 2.1% unexplained failures) into fast, explicit rejections that a client can handle.

Determining the limit required iteration. The initial value was 500. Running Experiment 3 against the updated image, BulkheadProbeUser — which expects a 503 when the server is full — returned all failures even at 800 simulated users, meaning the bulkhead never fired. Investigation via /health showed sse_clients peaking at ~85–296 per task regardless of user count; Locust's gevent runtime does not hold iter_content SSE connections open reliably when no data is buffered, so the server never actually reached 500 concurrent clients. The limit was lowered to 200: high enough to sit above what the test harness can naturally sustain (confirming the bulkhead fires correctly), low enough that the broadcast cost per anomaly event remains within the CPU budget observed in testing. With 200 in place, BulkheadProbeUser received consistent 503 responses and the enforcement was verified.

The load test suite was also expanded to cover the gaps identified: a fixed-load horizontal scaling comparison was added to Experiment 1; Experiment 4 was created for zero-wait burst testing of the worker pool; the Experiment 2 Go script was rewritten with concurrent goroutine phases and a P50/P95/P99 summary table; the Experiment 3 Locust file was updated with automated chaos injection via boto3 at the 4-minute mark and a BulkheadProbeUser to verify 503 enforcement at steady state.

Problems encountered: Deploying the updated image to ECS surfaced two issues back to back. The Docker image had been built on Apple Silicon (linux/arm64) but ECS Fargate requires linux/amd64 — tasks never started, with ECS reporting a platform manifest mismatch. Rebuilt with --platform linux/amd64. The ECR repository URL was also not in Terraform outputs, requiring a manual aws ecr describe-repositories query to get the push target.

After a successful deploy, the ECS rolling deployment retained old tasks while new ones started. This was confirmed when SSE client counts in the Experiment 3 re-run were exceeding the new 200-client bulkhead limit — the old image with the 500-client limit was still serving traffic. Old tasks were manually stopped via the ECS console once the new tasks passed health checks.